

Distributed Financial Transaction Processor

Complete Project Implementation Plan

Sampath Pranay Beela
Rahul China Jagadeesha

CS6650 — Building Scalable Distributed Systems

Task	Description	Owner(s)	Timeline	Dependencies	Deliverable
Project scope and architecture finalization	Finalize the system architecture for the distributed financial transaction processor, including API service, worker service, SQS, DynamoDB, ECS, and ALB.	Pranay, Rahul	Weeks 1–2	Project proposal approved	Architecture is documented and team responsibilities are clearly assigned.
Repository and service scaffolding	Set up the repository structure and scaffold the two Go services: <code>api-svc</code> and <code>worker-svc</code> .	Pranay	Weeks 1–2	Architecture finalized	Both services build and run locally with baseline endpoints and handlers.
DynamoDB schema design	Design the schema for account and transaction storage, including version attributes for optimistic concurrency control.	Rahul	Weeks 1–2	Architecture finalized	DynamoDB tables and fields are defined for balances, transactions, and conditional writes.
Core transfer workflow implementation	Implement end-to-end transfer logic from API request submission to worker-side processing and datastore update.	Pranay, Rahul	Weeks 1–2	Service scaffolding, schema design	A transfer can be submitted, processed, and reflected correctly in account balances.
Idempotency support	Add idempotency handling so repeated requests or duplicate SQS deliveries do not result in duplicate transfers.	Rahul	Weeks 1–2	Core transfer flow	Duplicate processing is prevented and repeated requests return consistent results.
SQS integration	Connect the API layer to SQS and the worker layer to queue consumption for asynchronous processing.	Pranay	Weeks 1–2	Core workflow	Requests are successfully enqueued and processed asynchronously by workers.
Local correctness verification	Build reconciliation logic to verify final balances after tests and ensure no money is lost or duplicated.	Rahul	Weeks 1–2	Core workflow, idempotency	Balance verification script confirms correctness after every run.
Containerization	Dockerize the services and set up local orchestration using Docker Compose.	Pranay	Weeks 3–4	Local services working	Entire system runs locally in containers with repeatable setup.
Terraform infrastructure setup	Write Terraform to provision AWS ECS, ALB, SQS, and DynamoDB resources for deployment.	Rahul	Weeks 3–4	Containerization complete	Infrastructure can be deployed reproducibly on AWS.
Cloud smoke testing	Deploy the system and run basic tests to confirm end-to-end transfer functionality in the deployed environment.	Pranay, Rahul	Weeks 3–4	Terraform complete	A sample transaction succeeds through the deployed system.
Load testing framework	Create Locust-based load tests for normal and high-contention transfer scenarios.	Pranay	Weeks 3–4	Core workflow complete	Load tests execute successfully and export throughput and latency results.
Experiment 1: Concurrent transfer storm	Simulate heavy contention by sending many concurrent transfers from the same account and measure conflict rate, retry rate, throughput, and p95 latency.	Pranay	Weeks 3–4	Load testing ready	Test completes with metrics captured and final balances verified as correct.
Experiment 2: Worker crash recovery	Kill a worker after it dequeues a message but before commit, then verify correctness under retries and redelivery.	Rahul	Weeks 3–4	SQS integration, idempotency	No duplicate or lost money after crash; recovery behavior is documented.

Milestone / Task	Description	Owner(s)	Timeline	Dependencies	Deliverable / Success Criteria
Pessimistic locking implementation	Add an alternative locking or coordination strategy to compare against optimistic locking.	Rahul	Weeks 5–6	Baseline system stable	Pessimistic locking path is functional and testable under the same workload.
Experiment 3: Optimistic vs. pessimistic comparison	Compare both concurrency control strategies under increasing load to evaluate retry rate, throughput, and latency tradeoffs.	Pranay, Rahul	Weeks 5–6	Pessimistic locking implemented	Comparative results identify where each strategy performs better.
Experiment 4: Horizontal scaling verification	Scale API and worker replicas from 1 to 2 to 4 while keeping workload fixed and verify both performance gains and correctness.	Pranay, Rahul	Weeks 5–6	AWS deployment stable	Scaling results are collected and correctness holds at each replica count.
Data analysis and chart generation	Analyze collected results and prepare charts for throughput, p95 latency, retry/conflict rate, and correctness outcomes.	Pranay	Weeks 7–8	All experiments complete	Clear charts and summaries are ready for inclusion in the final report.
Final report writing	Write the final report explaining system design, experiments, results, and architecture tradeoffs.	Rahul	Weeks 7–8	Analysis complete	Report is complete, coherent, and supported by experimental evidence.
Final demo preparation	Prepare a demo workflow showing the system, experiment highlights, and key findings.	Pranay, Rahul	Weeks 7–8	Report and experiments complete	Team is ready to present the project and explain major outcomes.